

Department of Electronic and
Telecommunication Engineering
University of Moratuwa

EN3150 - Pattern Recognition



Assignment 01
Learning from data and related challenges and linear
models for regression

230415H Mudaliarachchi N.S.

05/09/2026

Contents

1	Data Pre-processing and dealing with the Skewness	2
1.1	Answer	2
1.2	Answer	3
1.3	Answer	3
1.4	Answer	3
1.5	Answer	4
2	Linear Regression and Regularization	5
2.1	Answer	5
2.2	Answer	5
2.3	Answer	6
2.4	Answer	6
2.5	Answer	7
3	Robust Regression and the Huber Loss	8
3.1	Answer	8
3.2	Answer	8
3.3	Answer	8
4	Loss Functions Comparison	9
4.1	Answer	9
4.2	Answer	9
4.3	Answer	10
5	Code workings	10
6	References	10

1 Data Pre-processing and dealing with the Skewness

1.1 Answer

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.preprocessing import StandardScaler
4
5 def generate_features(n_samples=1000, student_id=123456):
6
7     np.random.seed(student_id%1000)
8
9     # feature 1: normal distribution data
10    feature_1 = np.random.normal(loc=50, scale=15, size=n_samples)
11
12    # feature 2: Exponential distribution data (highly skewed)
13    feature_2 = np.random.exponential(scale=2, size=n_samples)
14
15    # Include a few extreme artificial outliers in feature 2
16    feature_2[np.random.choice(n_samples, 5, replace=False)] = np.
        random.uniform(50, 100, 5)
17
18    return feature_1, feature_2
19
20 student_id = 230415
21 f1, f2 = generate_features(n_samples=1000, student_id=student_id)
22
23 plt.figure(figsize=(12, 5))
24 plt.subplot(1, 2, 1)
25 plt.hist(f1, bins=50, color='blue', alpha=0.7) # histogram for feature
    1
26 plt.title('Feature 1 Distribution')
27 plt.subplot(1, 2, 2)
28 plt.hist(f2, bins=50, color='orange', alpha=0.7) # histogram for
    feature 2
29 plt.title('Feature 2 Distribution')
30 plt.show()
```

1.2 Answer

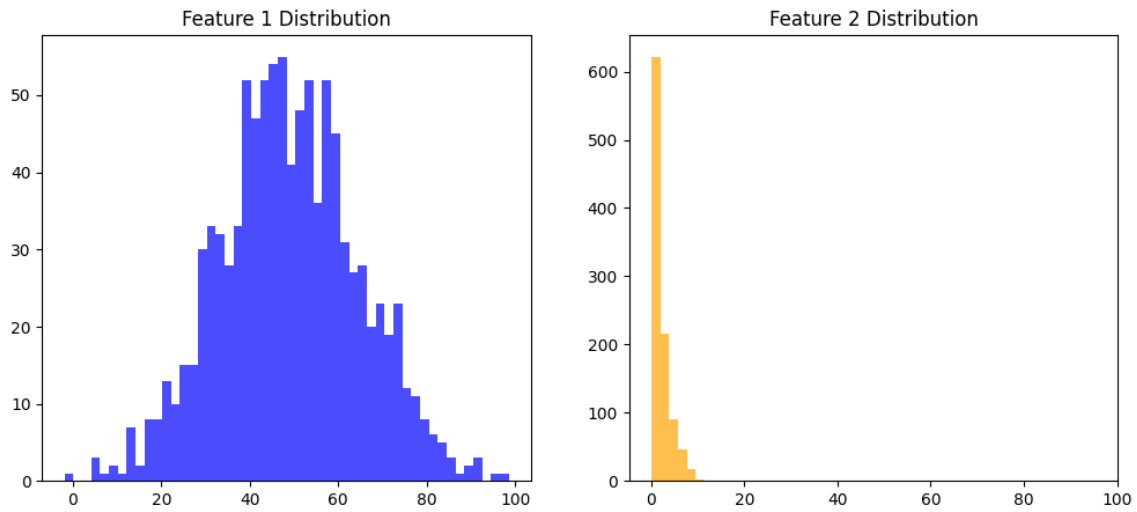


Figure 1: Histograms of both feature

1.3 Answer

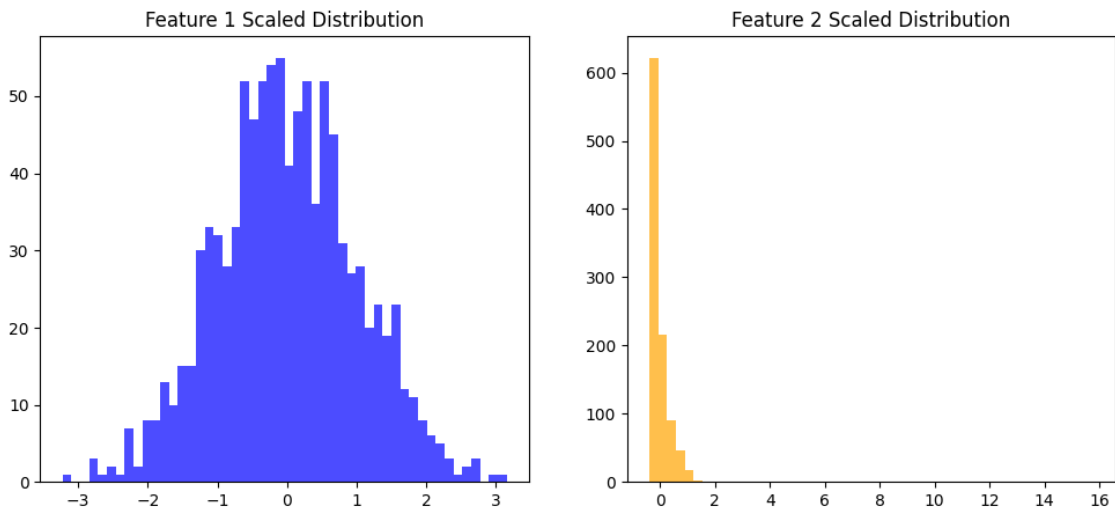


Figure 2: Z-score normalization for both of features

1.4 Answer

No.

Standard scaling doesn't normalize feature - 2 properly.

$$standard(x) = \frac{x - mean(x)}{std(x)} \quad (1)$$

In answer 1.2 figure 1 , as we can see feature 2 is right skewed.

For the feature 2 the data follows an exponential distribution and it contains several extreme outliers, between 50 to 100.(Answer 1 line no.16)

So the transformed feature has mean of 0 and standard deviation of 1. However the right skewed shape is remaining.

The mean and the standard deviation are sensitive to the outliers. Larger outlier values, increase the both of mean and standard deviation. But they are not removed or compressed by the transformation. As the result distribution remains highly skewed after scaling.

1.5 Answer

Since the feature is exponentially distributed, proper transformation is logarithmic transformation.

$$\ln(1 + x) \quad (2)$$

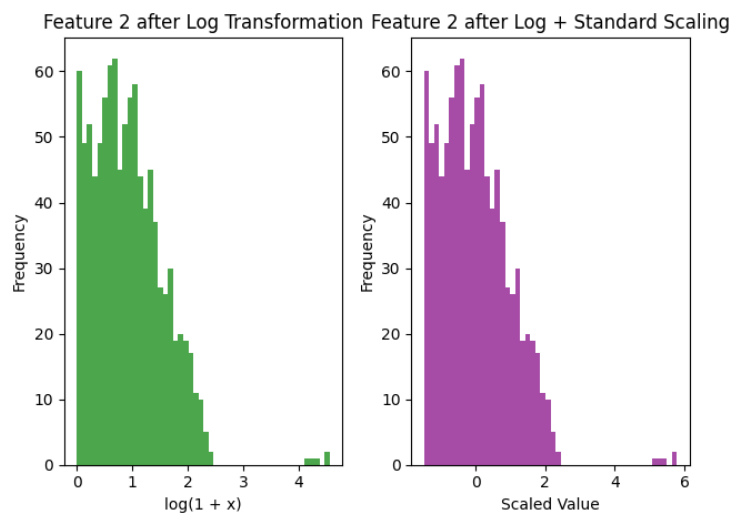


Figure 3: Caption

Feature 02 has larger values, logarithmic transformation will map those key values to small values. After applying standard, the distribution is set to mean = 0 and standard deviation to 1.

Effectiveness : The logarithmic scaling will reduced the skewness of the of the exponential distribution and compress the outliers. So this reduce the impact of the outliers.

2 Linear Regression and Regularization

2.1 Answer

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 from sklearn.datasets import load_diabetes
4 from sklearn import model_selection as model_selection
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import mean_squared_error, r2_score
7
8 # Load
9 diabetes_dataset = load_diabetes(as_frame=True)
10 # Split dataset
11 x_data, y_data = diabetes_dataset.data , diabetes_dataset.target
12
13 x_train, x_test, y_train, y_test = model_selection.train_test_split(
    x_data, y_data, test_size=0.2, random_state=42)
```

2.2 Answer

```
1 import numpy as np
2
3 y_predict = model.predict(x_test)
4
5 # Calculate RES
6 # df = number test of samples(n) - number of featues(p) -1
7 n = len(y_test)
8 print(f"Number of test samples: {n}")
9 p = x_test.shape[1]
10 print(f"Number of features: {p}")
11
12 df = n - p -1
13 print(f"Degree of freedom : {df}")
14
15 rss = np.sqrt((np.sum((y_test-y_predict)**2)/(df)))
16
17 mse = mean_squared_error(y_test, y_predict)
18
19 r2_stat = r2_score(y_test, y_predict)
20
21 print(f"Residual Standard Error : {rss:.2f}")
22 print(f"Mean Squred Error : {mse:.2f}")
23 print(f"R2 Statistic : {r2_stat:.2f}")
```

```
... Number of test samples: 89
    Number of features: 10
    Degree of freedom : 78
    Residual Standard Error : 57.53
    Mean Squred Error : 2900.19
    R2 Statistic : 0.45
```

Figure 4: MSE, RSE, R2 on testing set

2.3 Answer

```
1 from scipy.optimize import ridder
2 from sklearn import linear_model
3 from sklearn.linear_model import Ridge
4
5 # lamda = 0.1, 1.0, 10.0
6 lamda = [0, 0.1, 1.0, 10.0]
7 coefs = {}
8
9
10 for lmd in lamda:
11     rigde = Ridge(alpha=lmd)
12     rigde.fit(x_train, y_train)
13     coefs[lmd] = rigde.coef_
14
15 # plot
16 feature_indices = np.arange(x_train.shape[1])
17
18 plt.figure(figsize=(10, 5))
19 for lmd in lamda:
20     plt.plot(
21         feature_indices,
22         coefs[lmd],
23         "o--",
24         label=f"Ridge (  $\lambda$ ={lmd})",
25     )
26 plt.xlabel("Feature Index")
27 plt.ylabel("Estimated Coefficient Value (w)")
28 plt.title("Coefficient Magnitudes Across Models")
29 plt.xticks(feature_indices, [f"x{i+1}" for i in feature_indices])
30 plt.grid(True)
31 plt.legend()
32 plt.tight_layout()
33 plt.show()
```

2.4 Answer

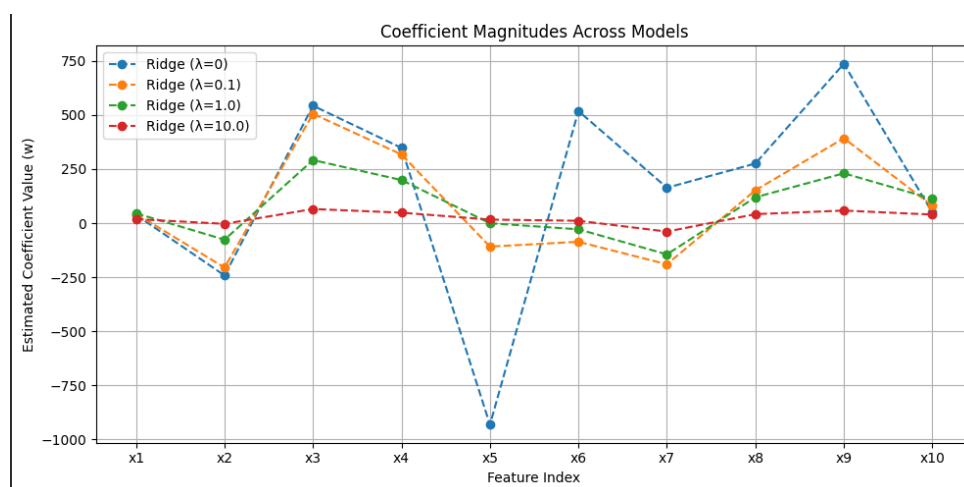


Figure 5: Magnitude of the estimated coefficients (w) for all four models

2.5 Answer

According to the plotted graph in section 2.4, $\lambda = 10$, all feature values are oscillating near zero. When λ is very large the estimated coefficient value goes to zero.

So this doesn't represent the correlation properly. In ridge regression, the objective function minimize the sum of squared errors subjected to an L2 penalty.

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - \mathbf{w}^T \mathbf{x}_i)^2 + \lambda \sum_{j=1}^d w_j^2 \quad (3)$$

$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad (4)$$

When $\lambda = 0$:

$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (5)$$

When λ goes to infinity :

$$\mathbf{w}_{\text{ridge}} = 0 \quad (6)$$

When features are highly co-related, the matrix in the equation 4, $\mathbf{X}^T \mathbf{X}$ becomes singular.

$$\det(\mathbf{X}^T \mathbf{X}) = 0 \quad (7)$$

$$\text{var}(\mathbf{w}_{\text{ridge}}) = \text{var}\{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}\} \quad (8)$$

$$\text{var}(\mathbf{w}_{\text{ridge}}) = \text{var}\{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{X} \mathbf{w} + \epsilon)\} \quad (9)$$

$$\text{var}(\mathbf{w}_{\text{ridge}}) = \text{var}\{\mathbf{W} + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \epsilon\} \quad (10)$$

$$\text{var}(\mathbf{w}_{\text{ridge}}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1} \quad (11)$$

How Ridge Regression solves this : By adding $\lambda \mathbf{I}$ to $\mathbf{X}^T \mathbf{X}$ Ridge forces all eigenvalues of the matrix to be strictly positive. This ensures the matrix is always invertible and well-conditioned, preventing the variance of coefficients from exploding.

3 Robust Regression and the Huber Loss

3.1 Answer

Model A has Lowest MSE.

$y_1 = 3.8x - 2.0$			$y_2 = 2.0x + 0.2$	
i	x_i	y_i	$(y_i - y_1)^2$	$(y_i - y_2)^2$
1	1	2.5	0.49	0.09
2	2	4.1	2.25	0.01
3	3	6.2	10.24	0
4	4	8	27.04	0.04
5	5	35	324	615.04
6	6	12.1	75.69	0.01
			73.285	102.5316667

Figure 6: standard Mean Squared Error (MSE) for both Model A and Model B

3.2 Answer

delta = 0.3							Model A	Model B
i	x_i	y_i	y_1	y_2	$ y_i - y_1 $		Huber Loss1	Huber Loss2
1	1	2.5	1.8	2.2	0.7		0.245	0.045
2	2	4.1	5.6	4.2	1.5		1.125	0.005
3	3	6.2	9.4	6.2	3.2		5.1	0
4	4	8	13.2	8.2	5.2		11.1	0.02
5	5	35	17	10.2	18		49.5	69.9
6	6	12.1	20.8	12.2	8.7		21.6	0.005
							14.77833333	11.6625

Figure 7: Huber Loss for both Model A and Model B using $\delta = 3.0$

Model B performs well.

3.3 Answer

Type of loss	y_1	y_2
MSE	324	615
Huber Loss	49.5	69.9

Table 1: Feature $\mathbf{X}_5$ in MSE and Huber Loss

In MSE, when outliers exist, the model heavily penalizes large errors due to the squared term $(y_i - \hat{y})^2$. As a result, the model deviates significantly toward the outlier points to minimize their influence.

Huber loss addresses this issue by using a piecewise function: it behaves like MSE for small errors ($|y_i - \hat{y}| \leq \delta$) and switches to a linear error $|y_i - \hat{y}|$ for large errors (outliers), reducing the outlier's influence on the model.

4 Loss Functions Comparison

4.1 Answer

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (12)$$

When $y_i = 1$:

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i)] \quad (13)$$

When $y_i = 0$:

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N \log(1 - \hat{y}_i) \quad (14)$$

The function, BCE mainly measures the distance between true labels and the predicted labels probabilities.

When $y_i = 1$:

- If $\hat{y}_i = 1$ (Correct prediction), then $\text{BCE} = 0$.
- If $\hat{y}_i = 0$ (False prediction), then BCE goes to infinity.

This penalizes the model exponentially as its predicted probability of event approaches zero when the event is actually present.

When $y_i = 0$:

- If $\hat{y}_i = 1$ (False prediction), then BCE goes to infinity.
- If $\hat{y}_i = 0$ (Correct prediction), then $\text{BCE} = 0$.

It penalizes the model exponentially as its predicted probability approaches 1 when the event is actually true.

4.2 Answer

Type of loss	error
MAE	0.99
BCE	4.6052

Table 2: Loss for a single positive sample ($y = 1$) when the model predicts $y = 0.01$ using both MAE and BCE.

4.3 Answer

MAE is linear. It gives loss of 0.99. This is bounded. The maximum possibility of any wrong prediction is steady clapped at 1.0.

BCE gives loss of 4.61, Because BCE use logarithmic scale. The predicted probability $\hat{y}_i$ the wrong extream loss approaches to infinity.

BCE heavily penalizes models that are confidently wrong.

5 Code workings

- Code : [google-colab](#)
- Question 3.1 and 3.2 workings : [google-sheet](#)

6 References

- Scikit-learn DOCS : [scikit-lear](#)
- Binary cross entropy : [geeksforgeeks](#)